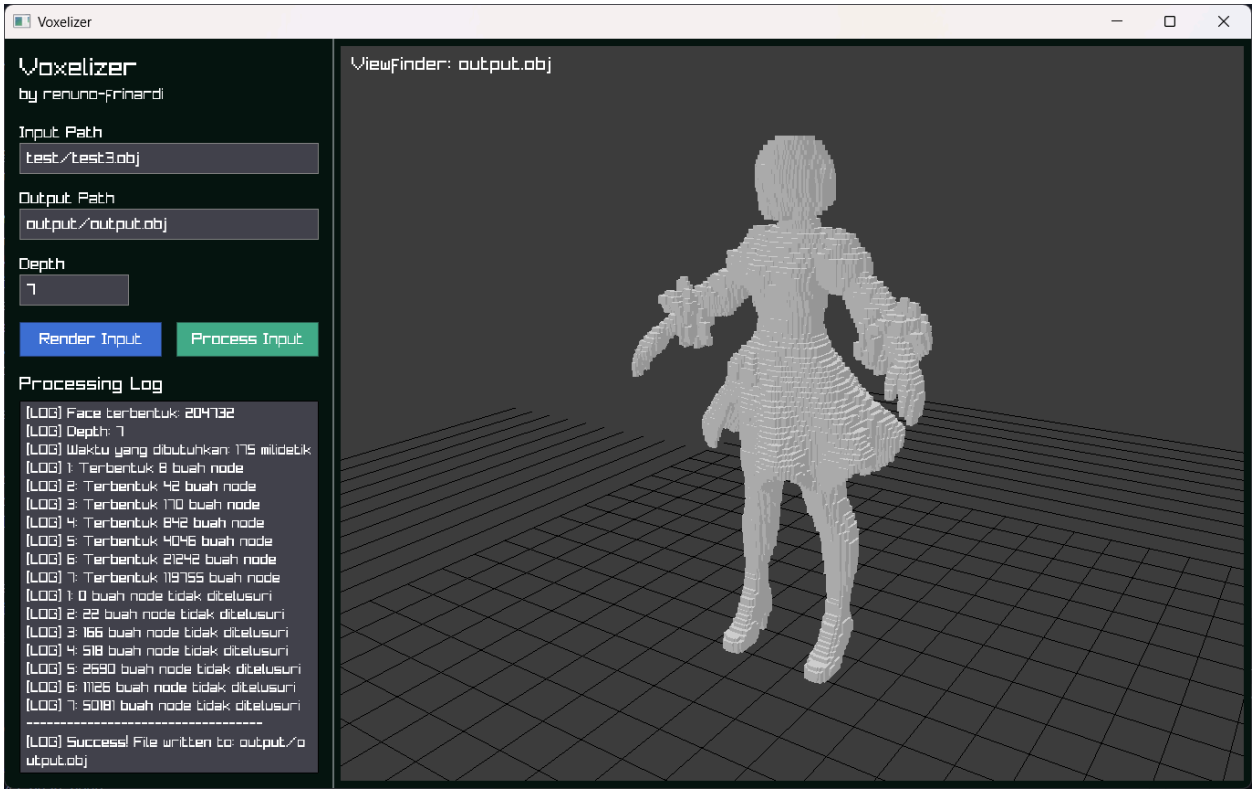
LAPORAN TUGAS KECIL II
IF2211 Strategi Algoritma
Semester II tahun 2025/2026

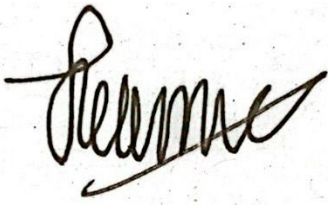


Voxelizer dengan Octre Menggunakan Pendekatan Algoritma *Divide and Conquer*

Renuno Yuqa Frinardi
13524080
Teknik Informatika

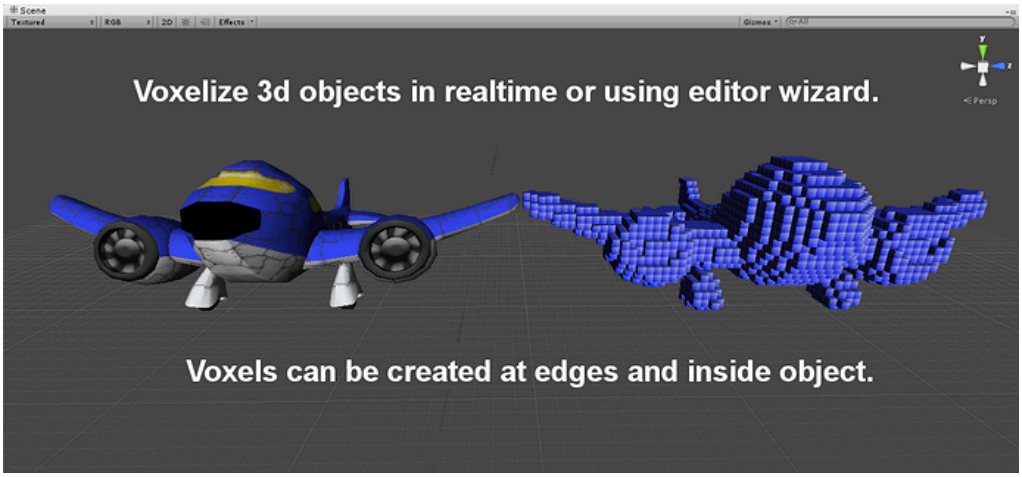
PERNYATAAN TIDAK MELAKUKAN KECURANGAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Renuno Yuqa Frinardi

I. Pendahuluan



Source: Unity Discussions

Voxelizer merupakan sebuah program atau alat yang berfungsi untuk melakukan konversi sebuah objek 3 dimensi menjadi susunan bentuk balok pada grid 3 dimensi atau kubus-kubus yang biasa disebut voxel. Di sini terdapat berbagai macam jenis algoritma yang dapat digunakan untuk implementasi sebuah program *voxelizer*. Salah satunya adalah pendekatan dengan menggunakan sebuah struktur data *Octree* atau pohon dengan 8 anak. Pendekatan ini digabungkan dengan *Axis-Aligned Bounding Box (AABB)* disertai pemikiran algoritma *Divide and Conquer* menghasilkan sebuah cara yang ampuh untuk menghadapi implementasi program ini.

II. Penjelasan Algoritma

A. Algoritma Utama

Fokus utama dari pendekatan *Divide and Conquer* pada program berfokus pada algoritma yang tertulis seperti berikut:

```
procedure subdivide(node, depth, maxDepth, vertices, faces,
nodePerDepth, leafPerDepth)
    if node = null then
        return

    nodePerDepth[depth] ← nodePerDepth[depth] + 1

    if depth >= maxDepth OR node.triangles.kosong then
        node.isLeaf ← true
        return

    childBoxes ← split_boundary(node.boundary)
    hasChild ← false
```

```
    i iterate [0..7]
        for triIndex in node.triangles do
            if intersects(triIndex, childBoxes[i], vertices, faces)
then
                childFaces.insert(triIndex

        if childFaces.kosong then
            leafPerDepth[depth + 1] ← leafPerDepth[depth + 1] + 1
            continue

        child ← new OctreeNode(childBoxes[i])
        child.triangles ← childFaces
        node.child[i] ← child
        node.isLeaf ← false
        hasChild ← true

        subdivide(child, depth + 1, maxDepth, vertices, faces,
nodePerDepth, leafPerDepth)

    if hasChild = false then
        node.isLeaf ← true
```

Setelah melakukan *chore* berupa membaca file .obj dan memetakan setiap *vertex* dan *face* dari objek ke sebuah struktur data, algoritma mulai memproses data dengan beberapa tahap seperti berikut:

1. **Inisiasi struktur data *Octree***

Program pertama-tama akan menginisiasi konstruktor untuk *Octre.* Program akan menerima seluruh face, vertex, juga *depth* dari *Octree* yang diinginkan dan menyimpannya dalam bentuk array/vektor (face dan vertex) juga *depth* dalam bentuk integer. Selain itu, dibuat juga sebuah *Axis-Aligned Bounding Box* yang bekerja dengan cara mengambil nilai maximum dan minimum x, y, dan z pada himpunan *vertex* dari objek *Octree*. Kedua koordinat tersebut akan membentuk sebuah balok yang akan merangkap objek dari file .obj di dalamnya. Setelah sebuah objek *Octree* ini terinisiasi, maka program akan melanjutkan ke langkah berikutnya, yaitu inisiasi proses rekursif.

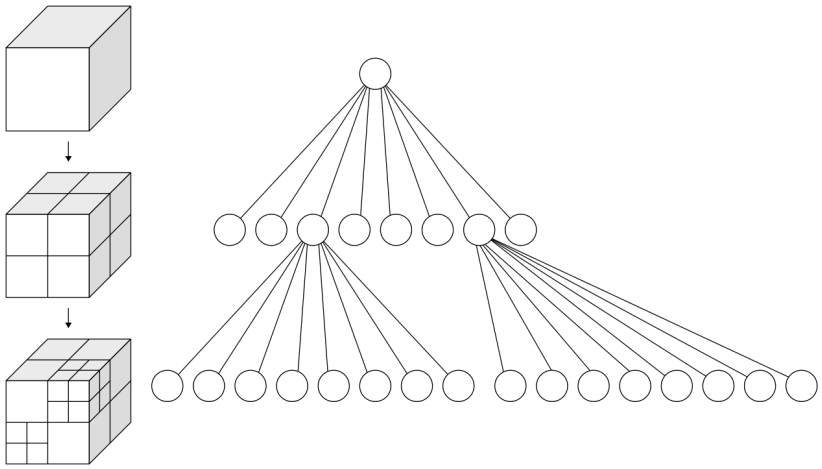
2. **Memanggil fungsi wrapper rekursif**

Pada tahap ini program akan memanggil fungsi *timer* terlebih dahulu yang akan menghitung lama keberjalanan program hingga file berhasill diproses. Setelah itu program akan memanggil sebuah fungsi rekursif untuk melakukan *subdivision* dari objek dengan menggunakan struktur data *octree*.

3. **Memulai proses rekursif *Divide and Conquer***

Pada tahap ini yang sama persis ada di *pseudocode* di atas, program akan mulai memproses data dan mengubah nya menjadi *voxelized object*. Di sini program akan menerima sebuah *depth* yang bermula dari nol, juga sebuah pointer ke object *octree*. Fungsi ini pertama tama akan mengecek apakah *depth* sudah melebihi batas maksimum yang diminta atau tidak, jika iya maka program berhenti. Jika tidak, maka program akan dilanjutkan untuk memproses data.

Objek root memiliki sebuah array Face yang berisi pola segitiga yang menyusun sebuah objek. Di sini program akan melakukan iterasi sebanyak 8 kali untuk membagi balok yang awalnya dibuat, menjadi 8 buah balok sama besar yang dibagi berdasarkan titik tengahnya. Setelah pembagian ini dijalankan, maka program akan mengecek seluruh himpunan *face* yang ada di root, yang beririsan ataupun berada di dalam sub-balok. Apabila *face* berada di dalam sub-balok tersebut, maka akan disimpan *index* dari *face* di dalam child node tersebut yang akan merefer ke *face* yang berkaitan di *root*. Setelah itu, fungsi akan memanggil kembali dirinya sendiri dengan menambah satu pada parameter *depth*.



Source: Eisenwave Github

Proses ini akan terus berulang sampai mencapai *depth* yang telah ditentukan, atau sampai dengan tidak ada lagi segitiga yang masuk ke dalam sub-balok. Program ini secara otomatis akan melakukan skip dan berfokus pada kubus-kubus yang memiliki *face* di dalamnya saja. Hasil akhir dari program ini merupakan seluruh *leaf* yang berada di *Octree*.

4. Menyimpann hasil dalam bentuk .obj

Setelah data diolah, leaf pada Octree dikonversi menjadi *voxel* dan disimpan sebagai *mesh* (daftar vertex dan face), yang kemudian diekspor ke file .obj. Untuk menghindari *face* internal yang memiliki duplikat, program menggunakan struktur *set* berisi kunci grid *voxel* 3D (ix, iy, iz) untuk menandai *voxel* yang terisi. Lalu untuk setiap voxel, program mengecek keberadaan tetangga pada 6 arah. Face hanya dibentuk pada sisi yang tidak memiliki tetangga, sehingga face yang saling berhimpitan tidak dimasukkan ke hasil akhir.

B. *Concurrency* [BONUS]

Diterapkan juga sebuah *concurrency*, di mana program akan melakukan *multithreading* dalam process subdivisi. *Multithreading* akan dilakukan maksimal pada *depth* kedua. Ini dilakukan untuk memitigasi *overthreading*, karena pada *depth* ke 6 saja ada kemungkinan di mana thread berjumlah lebih dari 262 ribu.

III. *Source Code* Program

Berikut adalah keseluruhan *source program* yang dibuat dalam bahasa pemrograman C++. Algoritma utama program dibagi menjadi 5 file sebagai berikut:

[geometry.h]

```
#pragma once

// Struktur untuk menyimpan Vertex dalam satu OBJ file
struct Vertex {
    float x, y, z;
};

// Struktur untuk menyimpan face dalam satu OBJ file
struct Face {
    int v1, v2, v3;
};
```

[aabb.hpp]

```
#pragma once

#include "geometry.h"
#include <algorithm>

// Struktur untuk menyimpan informasi Axis-Aligned Bounding Box (AABB)
struct AABB {
    Vertex min;
    Vertex max;

    // Inisiasi konstruktor dari AABB
    AABB() {}
    AABB(const Vertex& min, const Vertex& max) : min(min), max(max) {}
```

```
    // Fungsi untuk menghitung pusat dari AABB
    Vertex center() const {
        return {
            (min.x + max.x) * 0.5f,
            (min.y + max.y) * 0.5f,
            (min.z + max.z) * 0.5f
        };
    }

    // Fungsi untuk mengecek keteririsan antara dua AABB
    bool intersects(const AABB& other) const {
        return (min.x <= other.max.x && max.x >= other.min.x) &&
               (min.y <= other.max.y && max.y >= other.min.y) &&
               (min.z <= other.max.z && max.z >= other.min.z);
    }
};
```

[octree_node.hpp]

```
#pragma once

#include <vector>
#include <iostream>
#include "aabb.hpp"

// Kelas untuk node dari struktur Octree
class OctreeNode {
    private:
        // Properti yang dimiliki sebuah OctreeNode
        AABB boundary;
        std::vector<int> triangleIndices;
        OctreeNode* children[8];
        bool isLeaf;

    public:
        // Konstruktor dan destruktor dari OctreeNode
        OctreeNode(const AABB& boundary) {
        for (int i = 0; i < 8; i++) setChild(i, nullptr);
            if (!setBoundary(boundary) || !setIsLeaf(true) ||
```

```
!setTriangles({})) {
                printf("Error dalam inisasi AABB\n");
            }
        }
        ~OctreeNode() {
            for (int i = 0; i < 8; i++) {
                if (getChild(i) != nullptr) {
                    delete children[i];
                }
            }
        }

        // === Getter dan Setter untuk properti dari OctreeNode === //

        // Setter dan getter untuk triangleIndices
        bool setTriangles(const std::vector<int>& indices) {
            triangleIndices = indices;
            return true;
        }
        const std::vector<int>& getTriangles() const {
            return triangleIndices;
        }

        // Setter dan getter untuk boundary
        bool setBoundary(const AABB& newBoundary) {
            boundary = newBoundary;
            return true;
        }
        const AABB& getBoundary() const {
            return boundary;
        }

        // Setter dan getter untuk isLeaf
        bool setIsLeaf(bool val) {
            isLeaf = val;
            return true;
        }
        bool getIsLeaf() const {
            return isLeaf;
```

```
        }

        // Setter dan getter untuk children
        bool setChild(int i, OctreeNode* node) {
            children[i] = node;
            return true;
        }
        OctreeNode* getChild(int i) const {
            return children[i];
        }
};
```

[octree.hpp]

```
#pragma once

#include <vector>
#include <array>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <cstdio>
#include <future>
#include <mutex>
#include <set>
#include <chrono>
#include "geometry.h"
#include "aabb.hpp"
#include "octree_node.hpp"

// Kelas untuk struktur dari Octree
class Octree {
    private:
         // Properti yang dimiliki sebuah Octree
        OctreeNode* root;
        int maxDepth;
        mutable int totalVoxels;
        const std::vector<Vertex>& vertices;
        const std::vector<Face>& faces;
```

```
        std::vector<int> nodePerDepth;
        std::vector<int> leafPerDepth;
        std::chrono::steady_clock::time_point start;
        std::chrono::steady_clock::time_point end;
        std::chrono::milliseconds duration;

        // Helper untuk mengolah data OBJ dan proses vokseliasi
        void subdivide(OctreeNode* node, int depth);
        std::vector<AABB> split(const AABB& box);
        bool intersects(int faceIdx, const AABB& box);
        void collectLeaves(OctreeNode* node, std::vector<AABB>& boxes)
const;

    public:
        // Konstruktor dan destruktor dari Octree
        Octree(const std::vector<Vertex>& v,
            const std::vector<Face>& f,
            int maxDepth);
        ~Octree();

        // Method utama untuk membuat Octree, mendapatkan node terakhir,
dan melakukan generasi mesh akhir
        void build();
        std::vector<AABB> getLeafBoxes() const;
        void generateVoxelMesh(std::vector<Vertex>& outVertices,
std::vector<Face>& outFaces) const;
        int getNodeCountAtDepth(int depth) const;
        int getLeafCountAtDepth(int depth) const;
        int getTotalVoxels() const { return totalVoxels; }
        int getTimeTakenMs() const;
};
```

[octree.cpp]

```
#include "octree.hpp"

// Helper untuk MutEx lock dan grid cell
static std::mutex gDepthStatsMutex;
```

```
using GridCellKey = std::array<int, 3>;
static int coordToGridIndex(float value, float minValue, float step) {
    if (step == 0.0f) return 0;
    return (int)(std::lround((value - minValue) / step));
}

// Konstruktor dari Octree
Octree::Octree(const std::vector<Vertex> &v, const std::vector<Face> &f,
int maxDepth)
    : root(nullptr), maxDepth(maxDepth), vertices(v), faces(f),
totalVoxels(0) {
    if (vertices.empty()) {
        root = new OctreeNode(AABB(Vertex{0.0f, 0.0f, 0.0f},
Vertex{0.0f, 0.0f, 0.0f}));
        return;
    }

    Vertex min = vertices[0];
    Vertex max = vertices[0];

    for (size_t i = 0; i < vertices.size(); i++) {
        const Vertex &vert = vertices[i];
        min.x = std::min(min.x, vert.x);
        min.y = std::min(min.y, vert.y);
        min.z = std::min(min.z, vert.z);

        max.x = std::max(max.x, vert.x);
        max.y = std::max(max.y, vert.y);
        max.z = std::max(max.z, vert.z);
    }

    for (int i = 0; i <= maxDepth; i++) {
        nodePerDepth.push_back(0);
        leafPerDepth.push_back(0);
    }

    root = new OctreeNode(AABB(min, max));

    std::vector<int> allFaces;
```

```
    allFaces.reserve(faces.size());
    for (int i = 0; i < (int)(faces.size()); i++) {
        allFaces.push_back(i);
    }

    root->setTriangles(allFaces);
}

// Destruktor dari Octree
Octree::~Octree() { delete root; }

// Inisiasi proses subdivisi dari Octree
void Octree::build() {
    start = std::chrono::steady_clock::now();
    subdivide(root, 0);
    end = std::chrono::steady_clock::now();
    duration = std::chrono::duration_cast<std::chrono::milliseconds>(end
- start);
}

// Fungsi untuk melakukan subdivisi pada node dari Octree
void Octree::subdivide(OctreeNode *node, int depth) {
    if (!node) return;

    {
        std::lock_guard<std::mutex> lock(gDepthStatsMutex);
        if (depth >= 0 && depth < (int)(nodePerDepth.size()))
nodePerDepth[depth]++;
    }

    if (depth >= maxDepth || node->getTriangles().empty()) {
        return;
    }

    std::vector<AABB> boxes = split(node->getBoundary());

    std::vector<std::future<void>> futures;
    bool hasChild = false;
```

```
    for (int i = 0; i < 8; i++) {
        std::vector<int> childFaces;

        for (size_t j = 0; j < node->getTriangles().size(); j++) {
            int idx = node->getTriangles()[j];
            if (intersects(idx, boxes[i])) childFaces.push_back(idx);
        }

        if (childFaces.empty()) {
            std::lock_guard<std::mutex> lock(gDepthStatsMutex);
            if (depth >= 0 && depth < (int)(leafPerDepth.size()))
leafPerDepth[depth+1]++;
            continue;
        }

        OctreeNode *child = new OctreeNode(boxes[i]);
        child->setTriangles(childFaces);

        node->setChild(i, child);
        node->setIsLeaf(false);
        hasChild = true;

        if (depth < 3) futures.push_back(std::async(std::launch::async,
[this, child, depth]() { subdivide(child, depth + 1); }));
        else subdivide(child, depth + 1);
    }

    for (size_t i = 0; i < futures.size(); i++) futures[i].get();
    if (!hasChild) return;
}

// Helper untuk membagi AABB menjadi 8 sub bagian
std::vector<AABB> Octree::split(const AABB &b) {
    std::vector<AABB> result(8);
    Vertex m = b.center();

    for (int i = 0; i < 8; i++) {
        bool highX = (i & 4) != 0;
        bool highY = (i & 2) != 0;
```

```
        bool highZ = (i & 1) != 0;

        Vertex min;
        Vertex max;

        if (highX) {
            min.x = m.x;
            max.x = b.max.x;
        } else {
            min.x = b.min.x;
            max.x = m.x;
        }

        if (highY) {
            min.y = m.y;
            max.y = b.max.y;
        } else {
            min.y = b.min.y;
            max.y = m.y;
        }

        if (highZ) {
            min.z = m.z;
            max.z = b.max.z;
        } else {
            min.z = b.min.z;
            max.z = m.z;
        }

        result[i] = AABB(min, max);
    }

    return result;
}

// Helper untuk mengecek apakah sebuah face beririsan dengan AABB dari
node Octree
bool Octree::intersects(int faceIdx, const AABB &box) {
    const Face &f = faces[faceIdx];
```

```
    Vertex a = vertices[f.v1];
    Vertex b = vertices[f.v2];
    Vertex c = vertices[f.v3];

    Vertex center = {
        (a.x + b.x + c.x) / 3.0f,
        (a.y + b.y + c.y) / 3.0f,
        (a.z + b.z + c.z) / 3.0f
    };

    if (center.x >= box.min.x && center.x <= box.max.x &&
        center.y >= box.min.y && center.y <= box.max.y &&
        center.z >= box.min.z && center.z <= box.max.z)
        return true;

    Vertex tmin = {
        std::min({a.x, b.x, c.x}),
        std::min({a.y, b.y, c.y}),
        std::min({a.z, b.z, c.z})
    };

    Vertex tmax = {
        std::max({a.x, b.x, c.x}),
        std::max({a.y, b.y, c.y}),
        std::max({a.z, b.z, c.z})
    };

    AABB triBox(tmin, tmax);

    return box.intersects(triBox);
}

// Helper untuk mengumpulkan leaf dari Octree
void Octree::collectLeaves(OctreeNode *node, std::vector<AABB> &boxes)
const {
    if (!node) return;

    if (node->getIsLeaf()) {
```

```
        boxes.push_back(node->getBoundary());
        return;
    }

    for (int i = 0; i < 8; i++) collectLeaves(node->getChild(i), boxes);
}

// Method utama untuk mendapatkan leaf dari Octree
std::vector<AABB> Octree::getLeafBoxes() const {
    std::vector<AABB> boxes;
    collectLeaves(root, boxes);
    return boxes;
}

// Method untuk membuat mesh dari hasil vokselisasi
void Octree::generateVoxelMesh(std::vector<Vertex> &outVertices,
std::vector<Face> &outFaces) const {
    std::vector<AABB> leaves = getLeafBoxes();

    if (leaves.empty()) return;

    const AABB &rootBox = root->getBoundary();
    int gridSize = 1 << std::max(0, maxDepth);
    if (gridSize <= 0)gridSize = 1;

    float stepX = (rootBox.max.x - rootBox.min.x) / (float)(gridSize);
    float stepY = (rootBox.max.y - rootBox.min.y) / (float)(gridSize);
    float stepZ = (rootBox.max.z - rootBox.min.z) / (float)(gridSize);

    std::vector<GridCellKey> leafKeys;
    leafKeys.reserve(leaves.size());
    totalVoxels = (int)leaves.size();

    std::set<GridCellKey> occupied;
    for (size_t i = 0; i < leaves.size(); i++) {
        GridCellKey key = {
            coordToGridIndex(leaves[i].min.x, rootBox.min.x, stepX),
            coordToGridIndex(leaves[i].min.y, rootBox.min.y, stepY),
            coordToGridIndex(leaves[i].min.z, rootBox.min.z, stepZ),
```

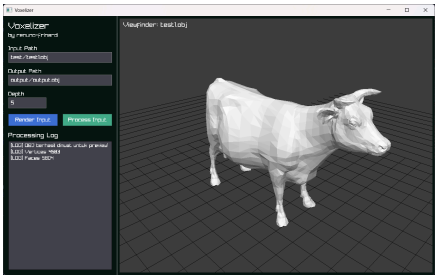
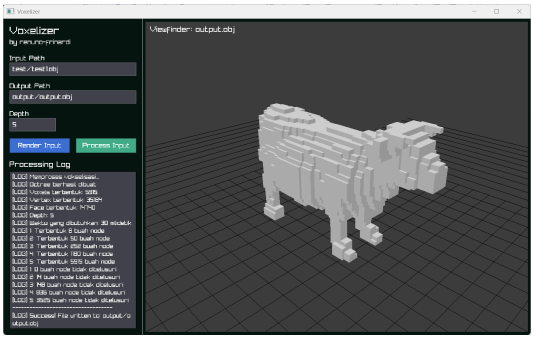
```
        };

        leafKeys.push_back(key);
        occupied.insert(key);
    }

    size_t surfaceVoxelCount = 0;

    for (size_t i = 0; i < leaves.size(); i++) {
        const GridCellKey &k = leafKeys[i];
        bool hasXp = occupied.count({k[0] + 1, k[1], k[2]}) > 0;
        bool hasXn = occupied.count({k[0] - 1, k[1], k[2]}) > 0;
        bool hasYp = occupied.count({k[0], k[1] + 1, k[2]}) > 0;
        bool hasYn = occupied.count({k[0], k[1] - 1, k[2]}) > 0;
        bool hasZp = occupied.count({k[0], k[1], k[2] + 1}) > 0;
        bool hasZn = occupied.count({k[0], k[1], k[2] - 1}) > 0;

        if (hasXp && hasXn && hasYp && hasYn && hasZp && hasZn)
continue;

        const AABB &box = leaves[i];
        int base = (int)outVertices.size();
        surfaceVoxelCount++;

        float x0 = box.min.x, y0 = box.min.y, z0 = box.min.z;
        float x1 = box.max.x, y1 = box.max.y, z1 = box.max.z;

        outVertices.push_back({x0, y0, z0});
        outVertices.push_back({x1, y0, z0});
        outVertices.push_back({x1, y1, z0});
        outVertices.push_back({x0, y1, z0});
        outVertices.push_back({x0, y0, z1});
        outVertices.push_back({x1, y0, z1});
        outVertices.push_back({x1, y1, z1});
        outVertices.push_back({x0, y1, z1});

        if (!hasZp) {
            outFaces.push_back({base + 4, base + 5, base + 6});
            outFaces.push_back({base + 4, base + 6, base + 7});
```

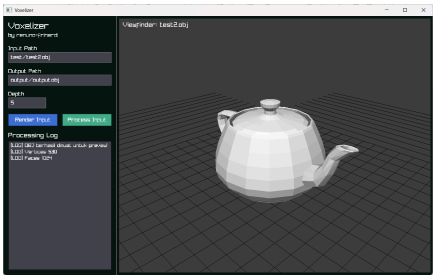
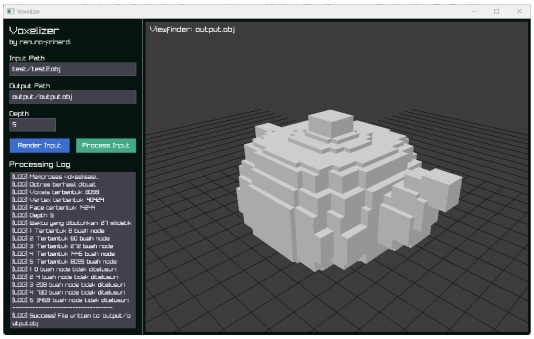
```
        }

        if (!hasZn) {
            outFaces.push_back({base + 1, base + 0, base + 3});
            outFaces.push_back({base + 1, base + 3, base + 2});
        }

        if (!hasXp) {
            outFaces.push_back({base + 5, base + 1, base + 2});
            outFaces.push_back({base + 5, base + 2, base + 6});
        }

        if (!hasXn) {
            outFaces.push_back({base + 0, base + 4, base + 7});
            outFaces.push_back({base + 0, base + 7, base + 3});
        }

        if (!hasYp) {
            outFaces.push_back({base + 3, base + 7, base + 6});
            outFaces.push_back({base + 3, base + 6, base + 2});
        }

        if (!hasYn) {
            outFaces.push_back({base + 0, base + 1, base + 5});
            outFaces.push_back({base + 0, base + 5, base + 4});
        }
    }

}

// Getter untuk mendapatkan jumlah node pada kedalaman tertentu
int Octree::getNodeCountAtDepth(int depth) const {
    if (depth < 0 || depth >= (int)(nodePerDepth.size())) return 0;
    return nodePerDepth[depth];
}

// Getter untuk mendapatkan jumlah node yang tidak ditelusuri pada
kedalaman tertentu
int Octree::getLeafCountAtDepth(int depth) const {
```

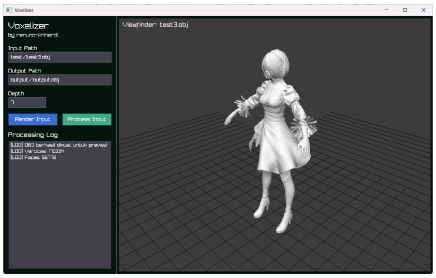
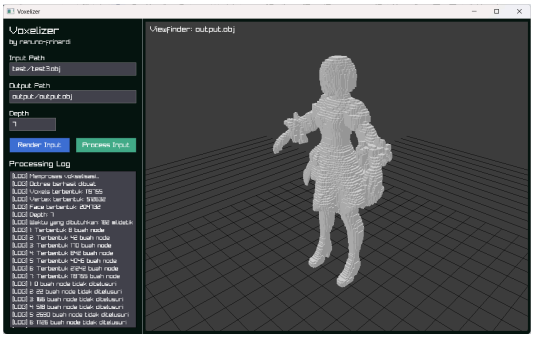
```
    if (depth < 0 || depth >= (int)(leafPerDepth.size())) return 0;
    return leafPerDepth[depth];
}

int Octree::getTimeTakenMs() const {
    return (int)duration.count();
}
```

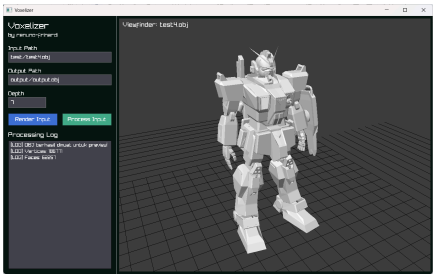
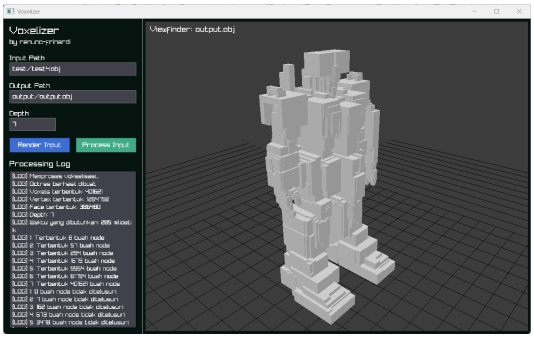
IV. Pengujian Program

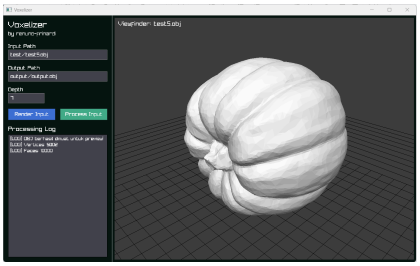
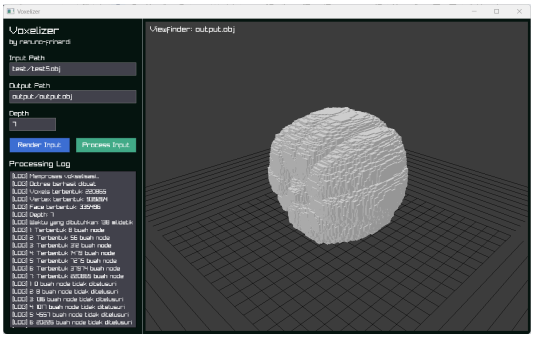
Berikut adalah kasus uji yang digunakan untuk mencoba aplikasi yang sudah dibuat

Input	Yang Diharapkan	Hasil
 Depth: 5 File: test1.obj	Sebuah file .obj yang sudah tervokselisasi dengan statistik yang sesuai	 [LOG] Memproses vokselisasi... [LOG] Octree berhasil dibuat [LOG] Voxels terbentuk: 5915 [LOG] Vertex terbentuk: 35184 [LOG] Face terbentuk: 14740 [LOG] Depth: 5 [LOG] Waktu yang dibutuhkan: 30 milidetik [LOG] 1: Terbentuk 8 buah node [LOG] 2: Terbentuk 50 buah node [LOG] 3: Terbentuk 252 buah node [LOG] 4: Terbentuk 1180 buah node [LOG] 5: Terbentuk 5915 buah node [LOG] 1: 0 buah node tidak ditelusuri [LOG] 2: 14 buah node tidak ditelusuri [LOG] 3: 148 buah node tidak ditelusuri [LOG] 4: 836 buah node tidak ditelusuri [LOG] 5: 3525 buah node tidak

		ditelusuri ---------- [LOG] Success! File written to: output/output.obj **Hasil sesuai**
 Depth: 5 File: test2.obj	Sebuah file .obj yang sudah tervokselisasi dengan statistik yang sesuai	 [LOG] Memproses vokselisasi... [LOG] Octree berhasil dibuat [LOG] Voxels terbentuk: 8099 [LOG] Vertex terbentuk: 40424 [LOG] Face terbentuk: 14244 [LOG] Depth: 5 [LOG] Waktu yang dibutuhkan: 27 milidetik [LOG] 1: Terbentuk 8 buah node [LOG] 2: Terbentuk 60 buah node [LOG] 3: Terbentuk 272 buah node [LOG] 4: Terbentuk 1446 buah node [LOG] 5: Terbentuk 8099 buah node [LOG] 1: 0 buah node tidak ditelusuri [LOG] 2: 4 buah node tidak ditelusuri [LOG] 3: 208 buah node tidak ditelusuri [LOG] 4: 730 buah node tidak ditelusuri [LOG] 5: 3469 buah node tidak ditelusuri ---------- [LOG] Success! File written to: output/output.obj **Hasil Sesuai**

 Depth: 7 File: test3.obj	Sebuah file .obj yang sudah tervokselisasi dengan statistik yang sesuai	 [LOG] Memproses vokselisasi... [LOG] Octree berhasil dibuat [LOG] Voxels terbentuk: 119755 [LOG] Vertex terbentuk: 512632 [LOG] Face terbentuk: 204732 [LOG] Depth: 7 [LOG] Waktu yang dibutuhkan: 182 milidetik [LOG] 1: Terbentuk 8 buah node [LOG] 2: Terbentuk 42 buah node [LOG] 3: Terbentuk 170 buah node [LOG] 4: Terbentuk 842 buah node [LOG] 5: Terbentuk 4046 buah node [LOG] 6: Terbentuk 21242 buah node [LOG] 7: Terbentuk 119755 buah node [LOG] 1: 0 buah node tidak ditelusuri [LOG] 2: 22 buah node tidak ditelusuri [LOG] 3: 166 buah node tidak ditelusuri [LOG] 4: 518 buah node tidak ditelusuri [LOG] 5: 2690 buah node tidak ditelusuri [LOG] 6: 11126 buah node tidak ditelusuri [LOG] 7: 50181 buah node tidak ditelusuri ---------- [LOG] Success! File written to: output/output.obj **Hasil Sesuai**

 Depth: 7 File: test4.obj	Sebuah file .obj yang sudah tervokselisasi dengan statistik yang sesuai	 [LOG] Memproses vokselisasi... [LOG] Octree berhasil dibuat [LOG] Voxels terbentuk: 401621 [LOG] Vertex terbentuk: 1254792 [LOG] Face terbentuk: 386480 [LOG] Depth: 7 [LOG] Waktu yang dibutuhkan: 285 milidetik [LOG] 1: Terbentuk 8 buah node [LOG] 2: Terbentuk 57 buah node [LOG] 3: Terbentuk 294 buah node [LOG] 4: Terbentuk 1679 buah node [LOG] 5: Terbentuk 9954 buah node [LOG] 6: Terbentuk 61794 buah node [LOG] 7: Terbentuk 401621 buah node [LOG] 1: 0 buah node tidak ditelusuri [LOG] 2: 7 buah node tidak ditelusuri [LOG] 3: 162 buah node tidak ditelusuri [LOG] 4: 673 buah node tidak ditelusuri [LOG] 5: 3478 buah node tidak ditelusuri [LOG] 6: 17838 buah node tidak ditelusuri [LOG] 7: 92731 buah node tidak ditelusuri ---------- [LOG] Success! File written to: output/output.obj **Hasil Sesuai**

 Depth: 7 File: test5.obj	Sebuah file .obj yang sudah tervokselisasi dengan statistik yang sesuai	 [LOG] Memproses vokselisasi... [LOG] Octree berhasil dibuat [LOG] Voxels terbentuk: 220865 [LOG] Vertex terbentuk: 939264 [LOG] Face terbentuk: 335496 [LOG] Depth: 7 [LOG] Waktu yang dibutuhkan: 138 milidetik [LOG] 1: Terbentuk 8 buah node [LOG] 2: Terbentuk 56 buah node [LOG] 3: Terbentuk 312 buah node [LOG] 4: Terbentuk 1479 buah node [LOG] 5: Terbentuk 7275 buah node [LOG] 6: Terbentuk 37974 buah node [LOG] 7: Terbentuk 220865 buah node [LOG] 1: 0 buah node tidak ditelusuri [LOG] 2: 8 buah node tidak ditelusuri [LOG] 3: 136 buah node tidak ditelusuri [LOG] 4: 1017 buah node tidak ditelusuri [LOG] 5: 4557 buah node tidak ditelusuri [LOG] 6: 20226 buah node tidak ditelusuri [LOG] 7: 82927 buah node tidak ditelusuri ---------- [LOG] Success! File written to: output/output.obj **Hasil Sesuai**

V. Lampiran

Repository GitHub:

- https://github.com/renuno-frinardi/Tucil2_13524080

Tabel keterangan tugas:

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✔	
2	Program berhasil dijalankan	✔	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✔	
4	Program informasi-informasi atau report	✔	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✔	
6	[Bonus] Program menggunakan *concurrency*	✔	
7	[Bonus] Program dapat menampilkan visualisasi .obj	✔	